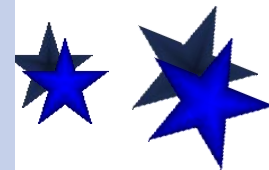


 **sel**


	0	1	2	3	4	5	6	7	8	9	10
1	0	0	1	1	1	1	1	1	1	1	1
2	0	0	0	0	1	1	1	1	1	1	1
3	0	0	0	0	0	0	0	0	1	1	1
4	0	0	0	0	0	0	0	1	0	1	0
5	0	0	0	0	0	0	1	1	1	1	1

物品4不装，说明前3个物品的容量为6，
查看`sel[3][6]=0`，说明不装入物品3；
继续查看`sel[2][6]=1`，说明装入物品2
前1个物品的容量=6-物品2的重量(2)=4
查看`sel[1][4]=1`，说明装入物品1。

```
i=n,j=W;  
while(i>0)  
    if(sel[i][j]==1) { printf("i");i--,j=j-w[i];}  
    else i--;
```

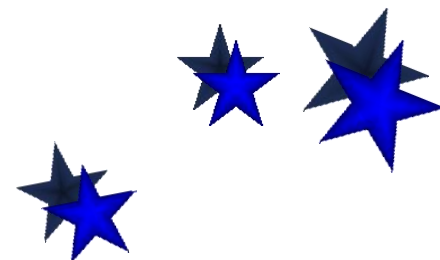


3.3 动态规划算法的典型应用

◆ **完全背包问题**：一个旅行者准备随身携带一个背包。可以放入背包的物品有 n 种，每种物品的重量和价值分别为 w_j, v_j ，**每种物品可以装入多个**。如果背包的最大重量限制是 b ，怎样选择放入背包的物品以使得背包的价值最大？

● 问题建模：

(1) 解是 $\langle x_1, x_2, \dots, x_n \rangle$ ，其中 x_i 是装入背包的第 i 种物品个数， x_i 为 ≥ 0 的整数。



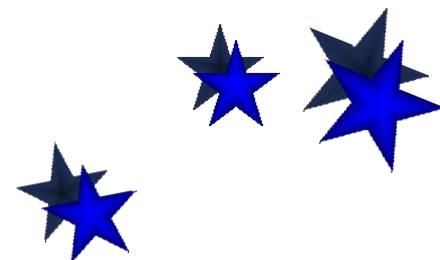
3.3 动态规划算法的典型应用

(2)

目标函数 $\max \sum_{i=1}^n v_i x_i$

约束条件 $\sum_{i=1}^n w_i x_i \leq b, \quad x_i \in \mathbb{N}$

- 线性规划问题: 由线性条件约束的线性函数取最大或最小的问题。
- 整数规划问题: 线性规划问题的变量 x_i 都是非负整数。



3.3 动态规划算法的典型应用

- 阶段划分

- 根据物品的个数，将问题的求解划分成 n 个阶段
- 第 k 个阶段决策出前 k 个物品在背包容量不超过 y 时的最大价值，记作： $F_k(y)$ ($0 \leq k \leq n$, $0 \leq y \leq b$)

- 子问题的边界

- $F_0(y)=0$ 不装入物品时，背包价值为0
- $F_k(0)=0$ 背包容量为0时，背包价值为0
- $F_k(y)=-\infty$ ($y < 0$) 背包容量为负数时，背包价值为 $-\infty$

- 状态转移方程

- 在决策 k 阶段，容量为 y 的最大价值时， $k-1$ 阶段以及 k 阶段小于 y 的子问题均已经解决。



3.3 动态规划算法的典型应用

➤ 递推方程1: $F_k(y) = \max \{F_{k-1}(y), F_k(y-w_k) + v_k\}$

没有装入
物品 k

k 阶段已对物品 k
决策完的结果。

说明: $F_k(y-w_k)$

(1) $y-w_k < 0$, 说明当前的 y 不够装入物品 k , 此时其值
 $F_k(y) = F_{k-1}(y)$ 。

(2) $y-w_k \geq 0$, 说明当前的 y 能够装入物品 k , 此时装入物品 k , 剩余容量 $y-w_k$ 分两种情况:

- 未曾装入物品 k , 则 $F_k(y-w_k) = F_{k-1}(y-w_k)$
- 已经装入过物品 k , 则 $F_k(y-w_k) \geq F_{k-1}(y-w_k)$
- 上述两种情况可统一为: $F_k(y-w_k) + v_k$



3.3 动态规划算法的典型应用

➤ 递推方程2(形式同资源分配问题)

$$F_k(y) = \max_{0 \leq x_k \leq \lfloor y/w_k \rfloor} \{F_{k-1}(y - x_k w_k) + x_k v_k\}$$

说明:

- (1) x_k 为物品 k 的装入个数, 从0直到 $\text{int}(y/w_k)$, 取价值最大
- (2) 写法1和写法2含义相同, 但效率相差很大
 - 写法1: 计算 $F_k(y)$ 时, $F_{k-1}(y)$ 和 $F_k(y-w_i)$ 已经计算完毕, 直接取值使用, 因此复杂度为 $O(1)$ 。
 - 写法2: 要经过 $\text{int}(y/w_k)$ 次循环才能得出 $F_k(y)$ 的值, 因此若 y 较大, w_k 较小时, 计算量也是很客观的。



3.3 动态规划算法的典型应用

- 解的追踪

(1) 使用一维数组 dp 存储第 k 阶段的决策结果。在第 k 阶段可先将决策结果暂存于 $temp$ 数组，该阶段结束后再更新 dp 数组中的内容。

(2) 使用二维数组 sel 实现。 $sel[k, y]$ 存储 k 阶段容量不超过 y 且取得最大价值时，**装入的最后一个物品编号**。例如：

- $sel[3, 4] = 2$ ，说明在第3阶段容量不超过4且价值最大时，最后装入的物品是2，即此时没有装入3号物品。
- $sel[4, 8] = 4$ ，说明在第4阶段容量不超过8且价值最大时，最后装入的物品是4。

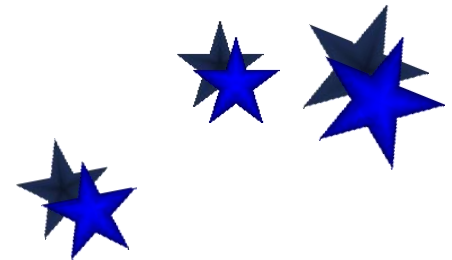


3.3 动态规划算法的典型应用

– $sel[k,y]$ 值的选取

$$sel[k,y] = \begin{cases} sel[k-1,y] & F_{k-1}(y) > F_k(y-w_k) + v_k \quad \text{没有装入物品 } k \\ k & F_{k-1}(y) \leq F_k(y-w_k) + v_k \quad \text{装入物品 } k \end{cases}$$

$$sel[1,y] = \begin{cases} 0 & y < w_1 \quad \text{装入物品 1} \\ 1 & y \geq w_1 \quad \text{装入物品 1} \end{cases}$$



3.3 动态规划算法的典型应用

- 实例 输入: $v_1=1, v_2=3, v_3=5, v_4=9,$
 $w_1=2, w_2=3, w_3=4, w_4=7,$
 $b=10$
- 数据结构
 - dp : 存储当前阶段求得的最大价值, 所有阶段结束后, $dp[10]$ 中即为最优值。
 - sel : 存储 k 阶段 y 容量取得最优值时, 装入的最后一个物品编号。
- 初始化

	0	1	2	3	4	5	6	7	8	9	10
dp	0	0	0	0	0	0	0	0	0	0	0



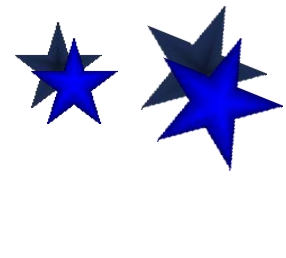
- * 第1阶段决策：前1个物品在背包容量不超过 j ($0 \leq j \leq 10$) 情况下的决策结果。 $w_1=2, v_1=1$

	0	1	2	3	4	5	6	7	8	9	10
dp	0	0	1	1	2	2	3	3	4	4	5

$$temp[j] = \max\{dp[j], temp[j-w_1] + v_1\} \quad 0 \leq j \leq 10$$

	0	1	2	3	4	5	6	7	8	9	10
temp	0	0	1	1	2	2	3	3	4	4	5

	0	1	2	3	4	5	6	7	8	9	10
1	0	0	1	1	1	1	1	1	1	1	1
2											
3											
4											



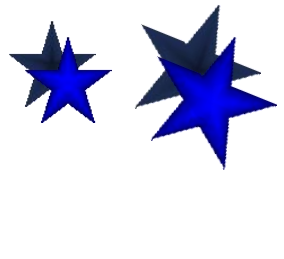
- * 第2阶段决策：前2个物品在背包容量不超过 j ($0 \leq j \leq 10$) 情况下的决策结果。 $w_2=3, v_2=3$

	0	1	2	3	4	5	6	7	8	9	10
dp	0	0	1	3	3	4	6	6	7	9	9

$$temp[j] = \max\{dp[j], temp[j-w_2] + v_2\} \quad 0 \leq j \leq 10$$

	0	1	2	3	4	5	6	7	8	9	10
temp	0	0	1	3	3	4	6	6	7	9	9

	0	1	2	3	4	5	6	7	8	9	10
1	0	0	1	1	1	1	1	1	1	1	1
2	0	0	1	2	2	2	2	2	2	2	2
3											
4											



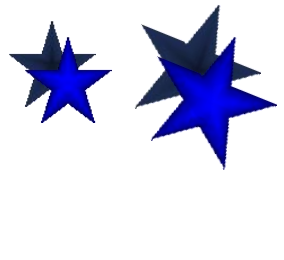
- * 第3阶段决策：前3个物品在背包容量不超过 j ($0 \leq j \leq 10$) 情况下的决策结果。 $w_3=4, v_3=5$

	0	1	2	3	4	5	6	7	8	9	10
dp	0	0	1	3	5	5	6	8	10	10	11

$$temp[j] = \max\{dp[j], temp[j-w_3] + v_3\} \quad 0 \leq j \leq 10$$

	0	1	2	3	4	5	6	7	8	9	10
temp	0	0	1	3	5	5	6	8	10	10	11

	0	1	2	3	4	5	6	7	8	9	10
1	0	0	1	1	1	1	1	1	1	1	1
2	0	0	1	2	2	2	2	2	2	2	2
3	0	0	1	2	3	3	3	3	3	3	3
4											



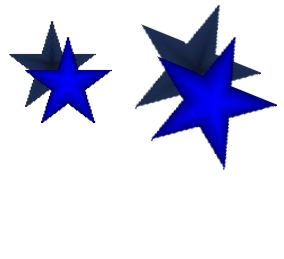
- * 第4阶段决策：前4个物品在背包容量不超过 j ($0 \leq j \leq 10$) 情况下的决策结果。 $w_4=7, v_4=9$

dp	0	1	2	3	4	5	6	7	8	9	10
	0	0	1	3	5	5	6	9	10	10	12

$$temp[j] = \max\{dp[j], temp[j-w_4] + v_4\} \quad 0 \leq j \leq 10$$

temp	0	1	2	3	4	5	6	7	8	9	10
	0	0	1	3	5	5	6	9	10	10	12

	0	1	2	3	4	5	6	7	8	9	10
1	0	0	1	1	1	1	1	1	1	1	1
2	0	0	1	2	2	2	2	2	2	2	2
3	0	0	1	2	3	3	3	3	3	3	3
4	0	0	1	2	3	3	3	4	3	4	4



➤ 解的输出

(1) 满足约束条件的最大价值为 $dp[10]=12$

	0	1	2	3	4	5	6	7	8	9	10
dp	0	0	1	3	5	5	6	9	10	10	12

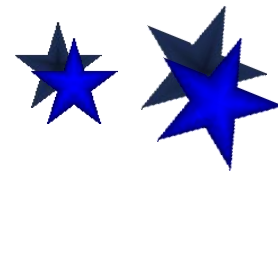
(2) 装入物品的输出

$sel[4][10]=4$, 说明最后一个装入物品4;

因物品4可能多次装入,

继续检查 $sel[4][10-w_4]=sel[4][3]$

	0	1	2	3	4	5	6	7	8	9	10
1	0	0	1	1	1	1	1	1	1	1	1
2	0	0	1	2	2	2	2	2	2	2	2
3	0	0	1	2	3	3	3	3	3	3	3
4	0	0	1	2	3	3	3	4	3	4	4





	0	1	2	3	4	5	6	7	8	9	10
1	0	0	1	1	1	1	1	1	1	1	1
2	0	0	1	2	2	2	2	2	2	2	2
3	0	0	1	2	3	3	3	3	3	3	3
4	0	0	1	2	3	3	3	4	3	4	4



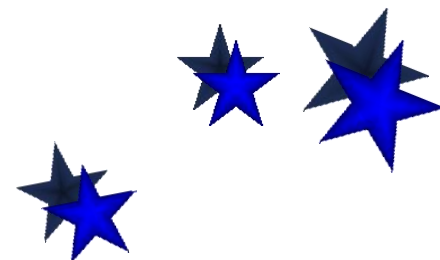
查看 $sel[4][3]=2$ ，说明该状态下，最后装入的物品编号是2号，即4号只装了1次、3号没有装。

因为2号物品也可能装入多个，

因此继续查看 $sel[2][3-w_2]=sel[2][0]=0$ ，

解的追踪结束。

背包中装入1个物品4和1个物品2



根据 sel 矩阵追踪解的算法

算法3.3 TrackSolution

输入: sel 表

输出: $x_1, x_2, \dots, x_n$, n 种物品的装入量

1. for $j=1$ to n do

2. $x_j \leftarrow 0$

将解向量初始化为0

3. $y \leftarrow b, j \leftarrow n$

4. $j \leftarrow sel[j, y]$

从 $sel[n, b]$ 开始追踪解

5. $x_j \leftarrow 1$

6. $y \leftarrow y - w_j$

7. while $sel[j, y] = j$ do

8. $y \leftarrow y - w_j$

物品 j 是否多次装入

9. $x_j \leftarrow x_j + 1$

10. if $sel[j, y] \neq 0$ goto 4

物品编号 $\neq 0$, 继续追踪



3.3 动态规划算法的典型应用

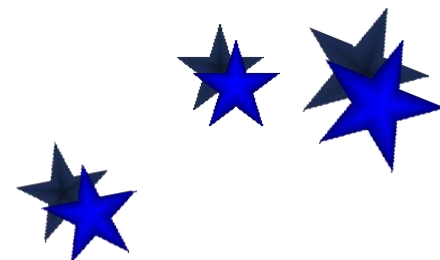
- 时间复杂度估算

根据公式

$$F_k(y) = \max\{F_{k-1}(y), F_k(y - w_k) + v_k\}$$

备忘录需计算 nb 项，每项常数时间，
计算时间为 $O(nb)$ 。

伪多项式时间算法：时间为参数 b 和 n 的多项式，不是输入规模的多项式。
参数 b 是整数，表达 b 需要 $\log b$ 位，
输入规模是 $\log b$ 。



3.3 动态规划算法的典型应用

◆ 最长公共子序列问题

- 子序列：设序列 X 和序列 Z ,

- $$X = \langle x_1, x_2, \dots, x_m \rangle$$

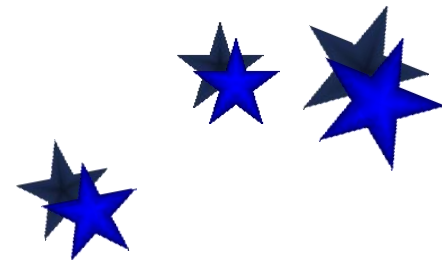
$$Z = \langle z_1, z_2, \dots, z_k \rangle$$

严格按下标递增序列从 X 中选出元素构成 $\langle x_{i_1}, x_{i_2}, \dots, x_{i_k} \rangle$

使得 $z_j = x_{i_j}, j = 1, 2, \dots, k$, 则称 Z 是 X 的子序列。

例如: $X = \{A, B, B, C, A, D, C\}$, $Z = \{A, B, D, C\}$

所以, Z 是 X 的子序列



3.3 动态规划算法的典型应用

- X 与 Y 的公共子序列 Z : Z 是 X 和 Y 的子序列。
- 子序列的长度: 子序列的元素个数
- 最长公共子序列问题

(1) 问题描述

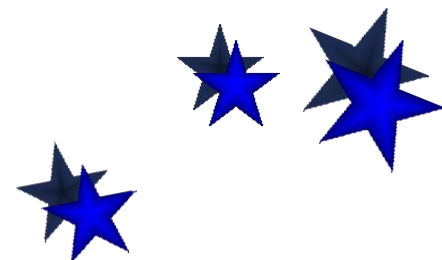
给定序列 $X = \langle x_1, x_2, \dots, x_m \rangle$, $Y = \langle y_1, y_2, \dots, y_n \rangle$
求 X 和 Y 的最长公共子序列。

(2) 实例

X : A B C B D A B

Y : B D C A B A

最长公共子序列: B C B A, 长度4



3.3 动态规划算法的典型应用

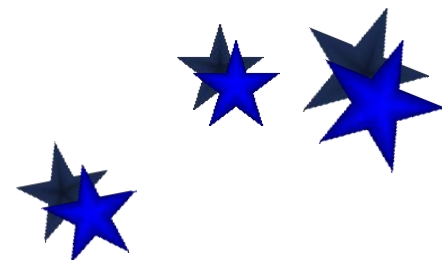
(3) 实现方法1: 蛮力算法

不妨设 $m \leq n$, $|X| = m$, $|Y| = n$

蛮力实现: 依次检查 X 的每个子序列在 Y 中是否出现。

- X 的每个子序列: 子集问题, m 个元素有 2^m 个子集
- 检查每个子序列在 Y 中是否出现过, 需要遍历 Y , 因此时间为 $O(n)$ 。

最坏情况下时间复杂度: $O(n 2^m)$



3.3 动态规划算法的典型应用

(4) 实现方法2：动态规划算法

□ 阶段划分及每个阶段中的状态结点

- 划分成 $i(1 \leq i \leq m)$ 个阶段，每个阶段决策 X 中的前 i 个字符
- 每个阶段包括 j 个状态结点($1 \leq j \leq n$)
- 每个状态结点对应： X 中的前 i 个字符与 Y 中前 j 个字符的最长公共子序列长度。

□ 边界状态（初始值）

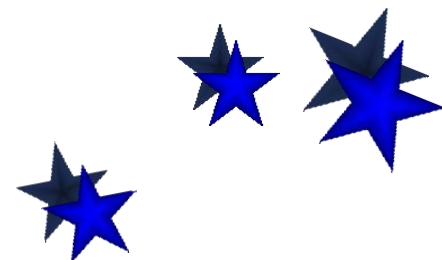
- $i=0$ 时，即 X 中的前0个字符，此时公共子序列长度为0
- $j=0$ 时，即 Y 中的前0个字符，此时公共子序列长度为0



3.3 动态规划算法的典型应用

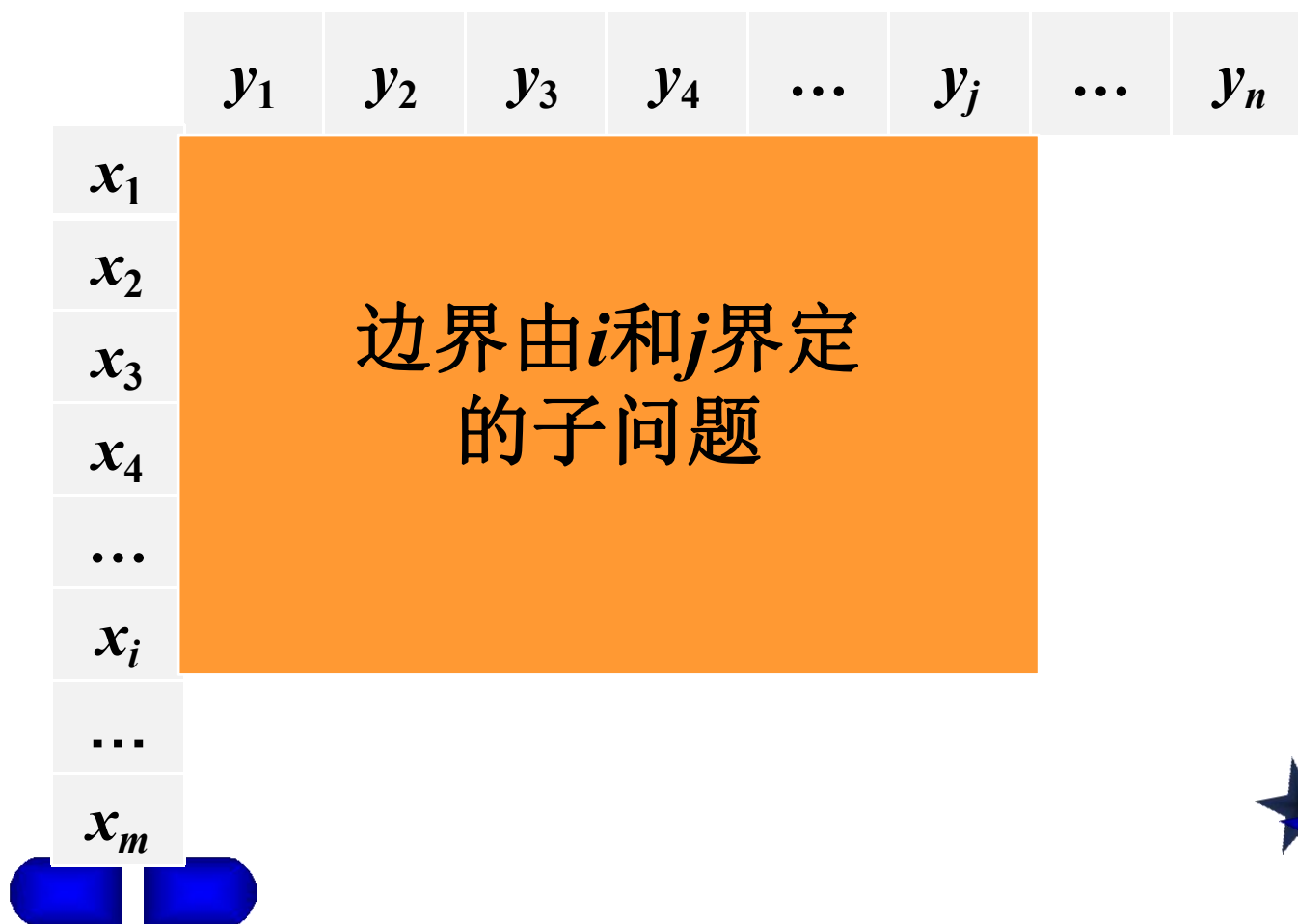
□ 状态转移方程

- 对 x 的前 i 个字符与 y 前 j 个字符进行决策
 - X 的前 $i-1$ 个字符在 Y 前 j 个字符的最长公共子序列已经决策结束 $F_{i-1}(j)$ ，可直接使用。
 - x 的前 i 个字符在 Y 前 $j-1$ 个字符的最长公共子序列已经决策结束 $F_i(j-1)$ ，可直接使用。
- 参数 i 和 j 界定子问题：
 - X 的终止位置是 i $(1 \leq i \leq m)$
 - Y 的终止位置是 j $(1 \leq j \leq n)$



3.3 动态规划算法的典型应用

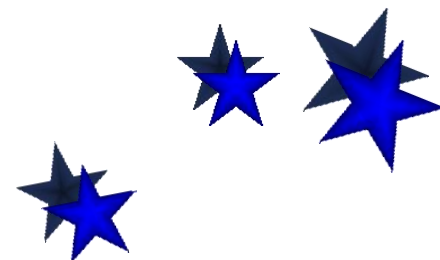
$$X_i = \langle x_1, x_2, \dots, x_i \rangle, \quad Y_j = \langle y_1, y_2, \dots, y_j \rangle$$



3.3 动态规划算法的典型应用

- 若子问题 X_i 和子问题 Y_j 的最长公共子序列为 Z_k :
 - $X_i = \langle x_1, x_2, \dots, x_i \rangle$, $Y_j = \langle y_1, y_2, \dots, y_j \rangle$,
 $Z_k = \langle z_1, z_2, \dots, z_k \rangle$
 - 情况1: $x_i = y_j$, 则 $z_k = x_i = y_j$, 即 Z_{k-1} 为 X_{i-1} 和 Y_{j-1} 的最长公共子序列, $F_i(j) = F_{i-1}(j-1) + 1$

	y_1	y_2	y_3	y_4	...	y_j
x_1	Z_{k-1}					
x_2						
x_3						
x_4						
...						
x_i						



3.3 动态规划算法的典型应用

– 情况2: $x_i \neq y_j$

(1) 若 $z_k \neq x_i$, 即 Z_{k-1} 为 X_{i-1} 和 Y_j 的最长公共子序列 $F_{i-1}(j)$

(2) 若 $z_k \neq y_j$, 即 Z_{k-1} 为 X_i 和 Y_{j-1} 的最长公共子序列 $F_i(j-1)$

(3) 此时, $F_i(j) = \max\{F_{i-1}(j), F_i(j-1)\}$

	y_1	y_2	y_3	y_4	...	y_j
x_1	(1) Z_{k-1}					
x_2						
x_3						
x_4						
...						
x_i						

	y_1	y_2	y_3	y_4	...	y_j
x_1	(2) Z_{k-1}					
x_2						
x_3						
x_4						
...						
x_i						



3.3 动态规划算法的典型应用

- 优化函数的递推方程

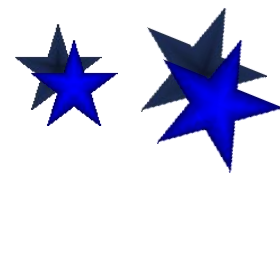
令 X 与 Y 的子序列

$$X_i = \langle x_1, x_2, \dots, x_i \rangle, \quad Y_j = \langle y_1, y_2, \dots, y_j \rangle$$

$C[i, j]$: X_i 与 Y_j 的 LCS 的长度

$C[i, j]$

$$= \begin{cases} 0 & \text{若 } i=0 \text{ 或 } j=0 \\ C[i-1, j-1]+1 & \text{若 } i, j > 0, x_i = y_j \\ \max\{C[i, j-1], C[i-1, j]\} & \text{若 } i, j > 0, x_i \neq y_j \end{cases}$$



3.3 动态规划算法的典型应用

– 数据结构设计

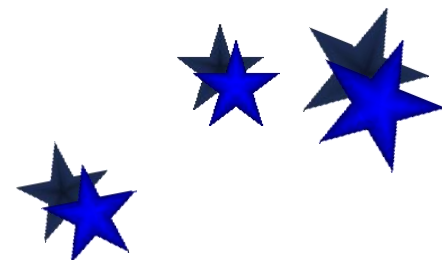
(1) ***dp***数组：存放*i*阶段各结点状态的最优值。因为在*i*阶段需要反复使用*i-1*阶段的结果，因此可将*i*阶段的值暂存于***temp***数组，该阶段结束后用***temp***替换***dp***中的最优值。

(2) 标记函数（数组）***M***用于追踪解。***M[i,j]***的值记录缩小子问题的方向，有三种情况：

✓ 若 $C[i,j]=C[i-1,j-1]+1$, $M[i,j]=\nwarrow$

✓ 若 $C[i,j]=C[i-1,j]$, $M[i,j]=\uparrow$

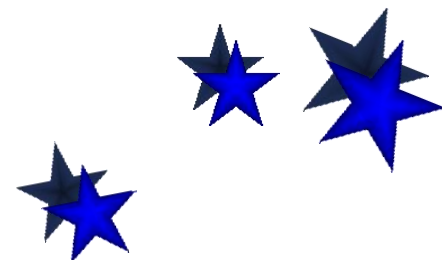
✓ 若 $C[i,j]=C[i,j-1]$, $M[i,j]=\leftarrow$



3.3 动态规划算法的典型应用

– 解的输出

- (1) $dp[n]$ 中即为最长公共子序列的长度值。
- (2) 使用数组 M 实现解的追踪，首先查看 $M[m, n]$,
 - × 若值=“↖”则输出对应字符 x_m ，继续在 $m-1, n-1$ 中追踪解。
 - × 若值=“↑”则继续在 $m-1, n$ 中追踪解。
 - × 若值=“←”则继续在 $m, n-1$ 中追踪解。



3.3 动态规划算法的典型应用

- 实例

输入: $X = \langle A, B, C, B, D, A, B \rangle$,
 $Y = \langle B, D, C, A, B, A \rangle$,

求解: 最长公共子序列及长度

数据结构设计:

(1) ***dp***: 存放当前阶段的最优解。 i 阶段求解过程中存放 $i-1$ 阶段的最优解, i 阶段决策结束后更新为 i 阶段最优解。

(2) ***temp***: 临时存放当前阶段的最优解

(3) ***M***: 标记数组, $M[i,j]$ 中存放 X 中前 i 个字符与 Y 中前 j 个字符最优解时, 缩小子问题的方向。



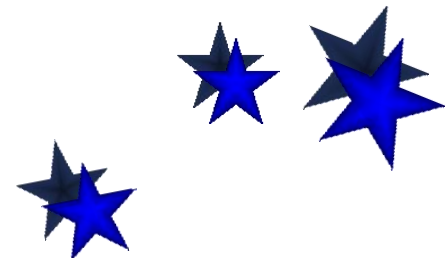
输入: $X = \langle A, B, C, B, D, A, B \rangle$,
 $Y = \langle B, D, C, A, B, A \rangle$,

➤ 初始化
 $i=0$

dp	0	1	2	3	4	5	6
	0	0	0	0	0	0	0

$temp$	0	1	2	3	4	5	6
	0						

M	1-B	2-D	3-C	4-A	5-B	6-A
1-A						
2-B						
3-C						
4-B						
5-D						
6-A						
7-B						



➤ 第一阶段($i=1$): x_1 与 $y_j(1 \leq j \leq 6)$

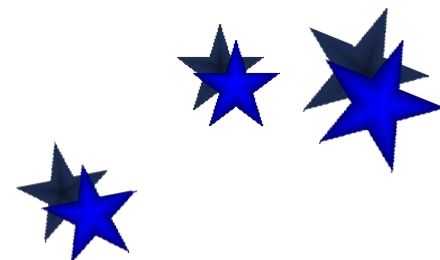
	dp	0	1	2	3	4	5	6
		0	0	0	0	0	0	0

$$x_1 = y_j \quad temp[j] = dp[j-1]$$

$$x_1 \neq y_j \quad temp[j] = \max\{dp[j], temp[i-1]\}$$

	0	1	2	3	4	5	6
$temp$	0	0	0	0	1	1	1

M	1-B	2-D	3-C	4-A	5-B	6-A
1-A	↑	↑	↑	↖	←	↖
2-B						
3-C						
4-B						
5-D						
6-A						
7-B						



➤ 第二阶段($i=2$): x_2 与 $y_j(1 \leq j \leq 6)$

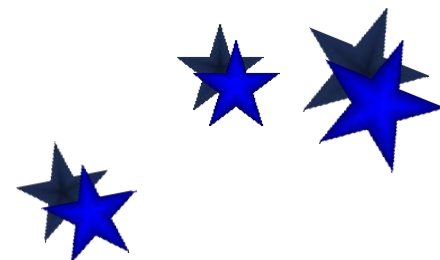
	dp	0	1	2	3	4	5	6
		0	0	0	0	1	1	1

$$x_2 = y_j \quad temp[j] = dp[j-1]$$

$$x_2 \neq y_j \quad temp[j] = \max\{dp[j], temp[i-1]\}$$

	0	1	2	3	4	5	6
$temp$	0	1	1	1	1	2	2

M	1-B	2-D	3-C	4-A	5-B	6-A
1-A	↑	↑	↑	↖	←	↖
2-B	↖	←	←	↑	↖	←
3-C						
4-B						
5-D						
6-A						
7-B						



➤ 第三阶段($i=3$): x_3 与 $y_j(1 \leq j \leq 6)$

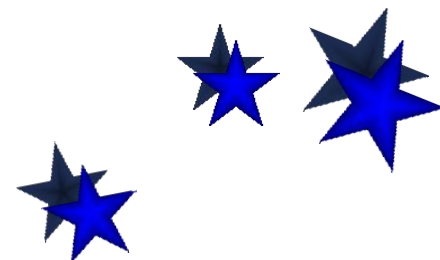
	dp	0	1	2	3	4	5	6
		0	1	1	1	1	2	2

$$x_3 = y_j \quad temp[j] = dp[j-1]$$

$$x_3 \neq y_j \quad temp[j] = \max\{dp[j], temp[i-1]\}$$

	0	1	2	3	4	5	6
$temp$	0	1	1	2	2	2	2

M	1-B	2-D	3-C	4-A	5-B	6-A
1-A	↑	↑	↑	↖	←	↖
2-B	↖	←	←	↑	↖	←
3-C	↑	↑	↖	←	↑	↑
4-B						
5-D						
6-A						
7-B						



➤ 第四阶段($i=4$): x_4 与 $y_j(1 \leq j \leq 6)$

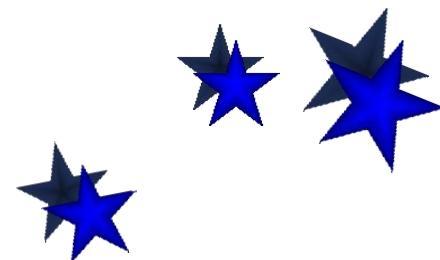
	dp	0	1	2	3	4	5	6
		0	1	1	2	2	2	2

$$x_4 = y_j \quad temp[j] = dp[j-1]$$

$$x_4 \neq y_j \quad temp[j] = \max\{dp[j], temp[i-1]\}$$

	0	1	2	3	4	5	6
$temp$	0	1	1	2	2	3	3

M	1-B	2-D	3-C	4-A	5-B	6-A
1-A	↑	↑	↑	↖	←	↖
2-B	↖	←	←	↑	↖	←
3-C	↑	↑	↖	←	↑	↑
4-B	↖	↑	↑	↑	↖	←
5-D						
6-A						
7-B						



➤ 第五阶段($i=5$): x_5 与 $y_j(1 \leq j \leq 6)$

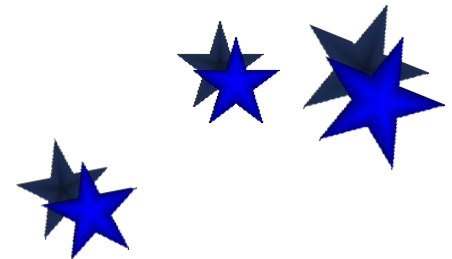
	dp	0	1	2	3	4	5	6
		0	1	1	2	2	3	3

$$x_5 = y_j \quad temp[j] = dp[j-1]$$

$$x_5 \neq y_j \quad temp[j] = \max\{dp[j], temp[i-1]\}$$

	0	1	2	3	4	5	6
$temp$	0	1	2	2	2	3	3

M	1-B	2-D	3-C	4-A	5-B	6-A
1-A	↑	↑	↑	↖	←	↖
2-B	↖	←	←	↑	↖	←
3-C	↑	↑	↖	←	↑	↑
4-B	↖	↑	↑	↑	↖	←
5-D	↑	↖	↑	↑	↑	↑
6-A						
7-B						



➤ 第六阶段($i=6$): x_6 与 $y_j(1 \leq j \leq 6)$

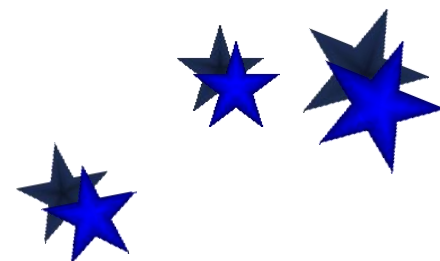
	dp	0	1	2	3	4	5	6
		0	1	2	2	2	3	3

$$x_6 = y_j \quad temp[j] = dp[j-1]$$

$$x_6 \neq y_j \quad temp[j] = \max\{dp[j], temp[i-1]\}$$

	0	1	2	3	4	5	6
$temp$	0	1	2	2	3	3	4

M	1-B	2-D	3-C	4-A	5-B	6-A
1-A	↑	↑	↑	↖	←	↖
2-B	↖	←	←	↑	↖	←
3-C	↑	↑	↖	←	↑	↑
4-B	↖	↑	↑	↑	↖	←
5-D	↑	↖	↑	↑	↑	↑
6-A	↑	↑	↑	↖	↑	↖
7-B						



➤ 第七阶段($i=7$): x_7 与 $y_j(1 \leq j \leq 6)$

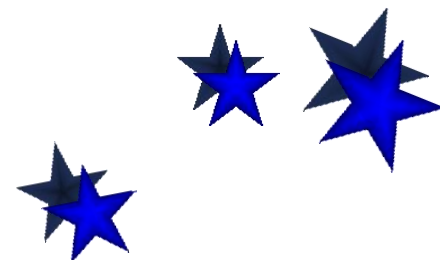
	dp	0	1	2	3	4	5	6
		0	1	2	2	3	3	4

$$x_7 = y_j \quad temp[j] = dp[j-1]$$

$$x_7 \neq y_j \quad temp[j] = \max\{dp[j], temp[i-1]\}$$

$temp$	0	1	2	3	4	5	6
	0	1	2	2	3	4	4

M	1-B	2-D	3-C	4-A	5-B	6-A
1-A	↑	↑	↑	↖	←	↖
2-B	↖	←	←	↑	↖	←
3-C	↑	↑	↖	←	↑	↑
4-B	↖	↑	↑	↑	↖	←
5-D	↑	↖	↑	↑	↑	↑
6-A	↑	↑	↑	↖	↑	↖
7-B	↖	↑	↑	↑	↖	↑



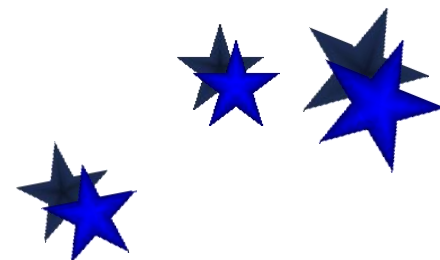
➤ 结果追踪:

	<i>dp</i>	0	1	2	3	4	5	6
		0	1	2	2	3	4	4

最长公共子序列长度

<i>M</i>	1-B	2-D	3-C	4-A	5-B	6-A
1-A	↑	↑	↑	↖	←	↖
2-B	↖	←	←	↑	↖	←
3-C	↑	↑	↖	←	↑	↑
4-B	↖	↑	↑	↑	↖	←
5-D	↑	↖	↑	↑	↑	↑
6-A	↑	↑	↑	↖	↑	↖
7-B	↖	↑	↑	↑	↖	↑

B C B A



最长公共子序列动态规划算法

■ 算法3.4 LCS(X, Y, m, n)

1. for $i \leftarrow 0$ to m do
2. $dp[i] \leftarrow 0$
3. for $i \leftarrow 1$ to m do
4. $temp[0] = 0$
5. for $j \leftarrow 1$ to n do
6. if $X[i] = Y[j]$
7. then $temp[j] \leftarrow dp[j-1] + 1$
8. $M[i, j] \leftarrow \nwarrow$
9. else if $dp[j] \geq temp[j-1]$
10. then $temp[j] \leftarrow dp[j]$
12. $M[i, j] \leftarrow \uparrow$
12. else $temp[j] \leftarrow temp[j-1]$
13. $M[i, j] \leftarrow \leftarrow$



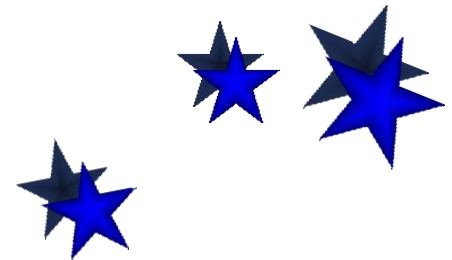
3.3 动态规划算法的典型应用

算法3.5 Structure Sequence(M, i, j)

输入: $M[i, j]$

输出: X 与 Y 的最长公共子序列

1. if $i=0$ or $j=0$ then return //一个序列为空
2. if $M[i, j] = “\searrow”$
3. then 输出 $X[i]$
4. **Structure Sequence**($B, i-1, j-1$)
5. else if $B[i, j] = “\uparrow”$ then **Structure Sequence** ($B, i-1, j$)
6. else **Structure Sequence** ($B, i, j-1$)



3.3 动态规划算法的典型应用

- 算法的时空复杂度

计算优化函数和标记函数：时间为 $\Theta(mn)$

赋初值，为 $O(n)$

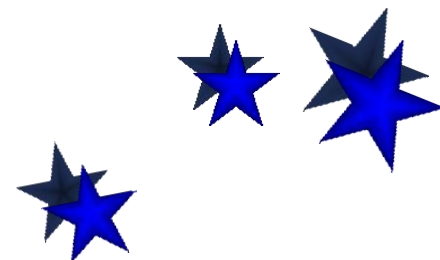
计算优化、标记函数迭代 $\Theta(mn)$ 次，
循环体内常数运算，

构造解：

每步缩小 X 或 Y 的长度，时间 $\Theta(m+n)$

算法时间复杂度： $\Theta(mn)$

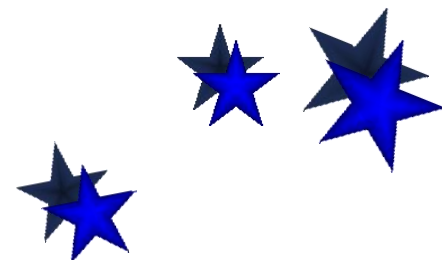
空间复杂度： $\Theta(mn)$



3.3 动态规划算法的典型应用

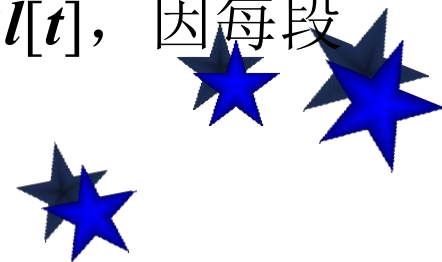
◆ 图像压缩存储

- 黑白图像的存储：存储每个像素点的灰度值：0—255，每个值最多8位二进制位。
- 图像的灰度值序列： $\{p_1, p_2, p_3, \dots, p_n\}$ ，其中 p_i 为第 i 个像素点的灰度值。
- 图像存储：每个像素的灰度值占8位，总计空间为 $8n$
- 问题：若有的图像大部分连续区域的灰度值都比较小，若均使用8位存储，则会造成很大的空间浪费。能否寻找一种更好的存储方式。



3.3 动态规划算法的典型应用

- 解决方案：图像变位压缩
 - (1) 变位压缩存储：将 $\{p_1, p_2, p_3, \dots, p_n\}$ 分成 m 段 $S_1, S_2, \dots, S_m$ 。
 - (2) 每段中含有的像素点个数可以不同，但同段中像素点存储位数一样，不同段中的像素点可以使用不同位数存储。
 - (3) 规定每段最多后存放256个像素点。
 - (4) 每个段增加段头，包括的信息
 - ①该段中包含像素点的个数， t 段该值记为 $l[t]$ ，因每段最多允许256个像素点，因此该值占8位。



3.3 动态规划算法的典型应用

②该段中各像素点占用的位数， t 段该值记为 $b[t]$ ，因每像素点最多8位，因此该值占3位。

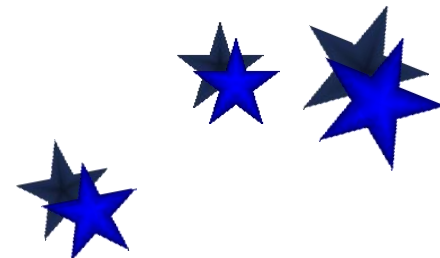
③因此：段头占据11位



$\{p_1 \dots p_i\}$ $\{p_{i+1} \dots p_j\}$ $\{p_{j+1} \dots p_k\}$ $\{p_{k+1} \dots p_n\}$

n 个像素点占用的总位数=

$$l[1] \times b[1] + l[2] \times b[2] + \dots + l[m] \times b[m] + 11m$$



3.3 动态规划算法的典型应用

➤ 图像压缩存储问题建模

- 输入：图像的 n 个像素点 $\{p_1, p_2, p_3, \dots, p_n\}$
- 输出：存储 n 个像素点的最少二进制位数以及分段的位置。

- 约束条件：第 t 段像素个数 $l[t] \leq 256$

第 t 段占用空间： $b[t] \times l[t] + 11$

$$b[t] = \left\lceil \log(\max_{p_k \in S_t} p_k + 1) \right\rceil \leq 8$$

按照段中最大值确定存储位数

- 目标函数： $\min_T \{ \sum_{t=1}^m (b[t] \times l[t] + 11) \},$
 $T = \{S_1, S_2, \dots, S_m\}$ 为分段



3.3 动态规划算法的典型应用

➤ 实例 灰度值序列

$P=\{10,12,15,255,1,2,1,1,2,2,1,1\}$

分法1: $S_1=\{10, 12, 15\}$, $S_2=\{255\}$,
 $S_3=\{1, 2, 1, 1, 2, 2, 1, 1\}$

分法2: $S_1=\{10,12,15,255,1,2,1,1,2,2,1,1\}$

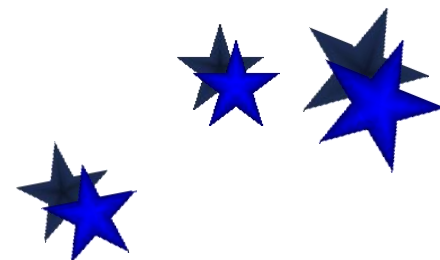
分法3: 分成12组, 每组一个数

存储空间

分法1: $11 \times 3 + 4 \times 3 + 8 \times 1 + 2 \times 8 = 69$

分法2: $11 \times 1 + 8 \times 12 = 107$

分法3: $11 \times 12 + 4 \times 3 + 8 \times 1 + 1 \times 5 + 2 \times 3 = 163$



3.3 动态规划算法的典型应用

➤ 子问题界定与计算顺序

子问题前边界为1，后边界为 i

对应像素序列为 $\langle p_1, p_2, \dots, p_i \rangle$

优化函数值 $S[i]$ 中存储子问题 i 的最优分段存储位数。

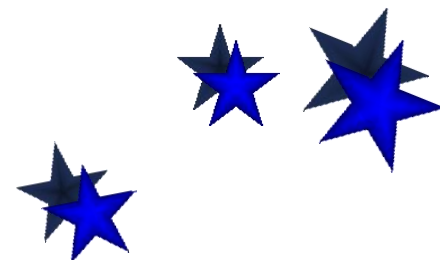
计算顺序

$i = 1$ 

$i = 2$ 

...

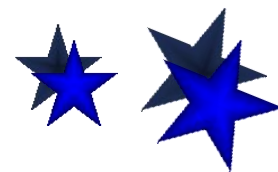
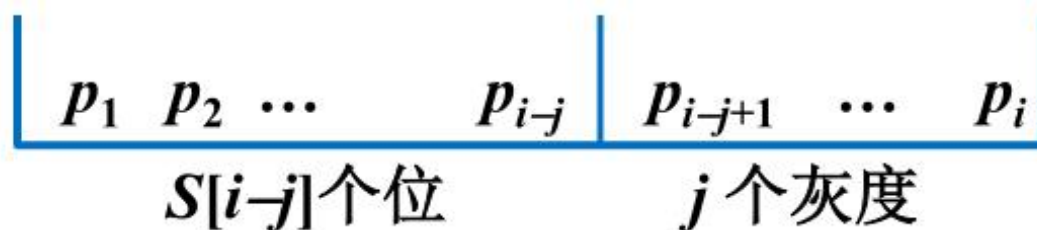
$i = n$ 



3.3 动态规划算法的典型应用

➤ 状态转移方程

- 在对子问题 i 进行决策时，子问题0到子问题 $i-1$ 的结果已经存储到 $S[0]$ 、 $S[1]$ $S[i-1]$ 中。
- 问题 i 的最后一个分段 S_m 的长度为 j ($1 \leq j \leq \min(i, 256)$)
 - (1) 前 $m-1$ 个分段中共有 $i-j$ 个像素点，其解已经存储到 $S[i-j]$ 中，可直接使用。
 - (2) S_m 段中有 j 个像素点，每个像素点的位数由 j 个像素点的最大值决定。



3.3 动态规划算法的典型应用

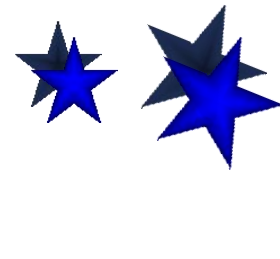
(2) 如何确定分段 S_m 占用的位数？

① S_m 长度=1时， S_m 中只含有 p_i ，计算其存储位数，记入 $b[i]$ 中。

② S_m 长度=2时， S_m 中含有 $p_{i-1}p_i$ ，计算 p_{i-1} 的存储位数并与 $b[i]$ 比较，将其中较大者记入 $b[i-1]$ 。

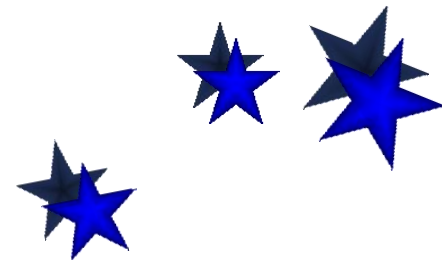
③ S_m 长度= j 时， S_m 中含有 $p_{i-j+1}\cdots p_{i-1}p_i$ ，计算 p_{i-j+1} 的存储位数并与 $b[i-j+2]$ 比较，将其中较大者记入 $b[i-j+1]$ 。

- 在对不同长度的 S_m 进行决策时，应事先计算该段包含像素的最大存储位数，并将其存储于 $b[i-j+1]$ 中。



3.3 动态规划算法的典型应用

- 数组***b***中的内容，不同阶段值是不同的，例如
 - (1) 第1阶段(p_1): 仅使用***b***[1]，存储 p_1 的位数。
 - (2) 第2阶段(p_1, p_2): 使用***b***[1]、***b***[2], ***b***[2]中存储 p_2 的位数, ***b***[1]中存储 $\max\{p_1, p_2\}$ 。
 - (3) 第*i*阶段($p_1 \dots p_i$): 使用***b***[1].....***b***[*i*], ***b***[*i*]中存储 p_i 的位数, ***b***[*j*]中存储 $\max\{p_{i-j+1} \dots p_i\}$, ***b***[1]中存储 $\max\{p_1 \dots p_i\}$ 。



3.3 动态规划算法的典型应用

- 求解子问题 i 的过程:

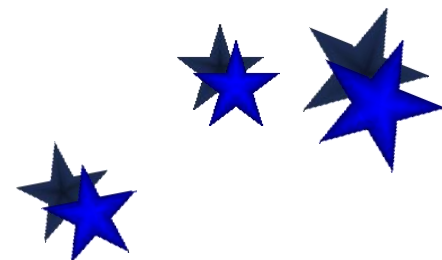
	S_m 长度	S_m 的位数	子问题 $i-j$	结果
$S[i]=$ min	1	$b[i]+11$	$S[i-1]$	$S[i-1]+b[i]+11$
	2	$2*b[i-1]+11$	$S[i-2]$	$S[i-2]+2*b[i]+11$
				
	j	$j*b[i-j+1]+11$	$S[i-j]$	$S[i-j]+j*b[i-j+1]+11$
				
	i	$n*b[1]+11$	$S[0]$	$S[0]+n*b[1]+11$

$$S[i]=\min\{S[i-j]+j*b[i-j+1]\}+11$$
$$1\leq j\leq\min(i,256)$$



3.3 动态规划算法的典型应用

- 实例: $P = \langle 10, 12, 15, 255, 1, 2 \rangle$
- 使用的数据结构:
 - (1) 数组 S : 存储每阶段求出的最优解, 例如 $S[i]$ 存储 $p_1 \cdots p_i$ 的最优解。
 - (2) 数组 b : 例如 $b[i-j+1]$ 中存储每阶段计算出的 $p_j \cdots p_i$ 每个像素点存储需要的位数。
 - (3) 数组 l : 标记函数 (追踪解), 用于记录分段的位置。
例如: 若 $l[i]$ 中记录的值为 x , 则说明最后一个段应包括 x 个像素点, 即 $p_{i-x+1} \cdots p_i$ 。



3.3 动态规划算法的典型应用

$P = \langle 10, 12, 15, 255, 1, 2 \rangle$

(1) 第一阶段(10)

$$b[1]=4$$

$$S[1]=\min\{S[0]+1*4\}+11=15$$

$$l[1]=1$$

S	15	19				
l	1	2				

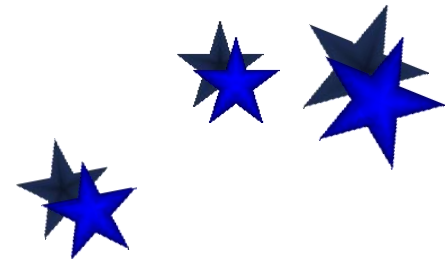
(2) 第一阶段(10, 12)

$$j=1 \quad b[2]=4 \quad S[1]+1*4=19$$

$$j=2 \quad b[1]=4 \quad S[0]+2*4=8$$

$$S[2]=\min\{19+8\}+11=19$$

$$l[2]=2$$



3.3 动态规划算法的典型应用

(3) 第三阶段 (10, 12, 15)

$$j=1 \quad b[3]=4 \quad S[2]+1*4=23$$

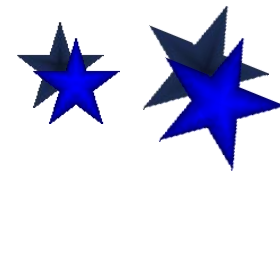
$$j=2 \quad b[2]=4 \quad S[1]+2*4=23$$

$$j=3 \quad b[3]=4 \quad S[0]+3*4=12$$

$$S[3]=\min\{23, 23, 12\}+11=23$$

$$l[3]=3$$

S	15	19	23			
l	1	2	3			



3.3 动态规划算法的典型应用

(4) 第四阶段 (10, 12, 15, 255)

$$j=1 \quad b[4]=8 \quad S[3]+1*8=31$$

$$j=2 \quad b[3]=8 \quad S[2]+2*8=35$$

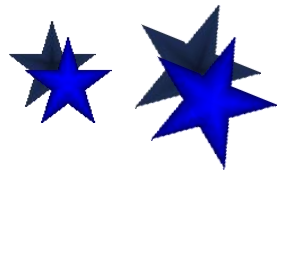
$$j=3 \quad b[2]=8 \quad S[1]+3*8=39$$

$$j=4 \quad b[1]=8 \quad S[0]+4*8=32$$

$$S[4]=\min\{31,35,39,32\}+11=42$$

$$l[4]=1$$

S	15	19	23	42		
l	1	2	3	1		



3.3 动态规划算法的典型应用

(5) 第五阶段 (10, 12, 15, 255, 1)

$$j=1 \quad b[5]=1 \quad S[4]+1*1=43$$

$$j=2 \quad b[4]=8 \quad S[3]+2*8=39$$

$$j=3 \quad b[3]=8 \quad S[2]+3*8=43$$

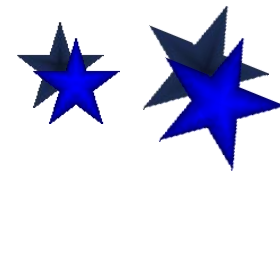
$$j=4 \quad b[2]=8 \quad S[1]+4*8=47$$

$$j=5 \quad b[1]=8 \quad S[0]+5*8=40$$

$$S[5]=\min\{43,39,43,47,40\}+11=50$$

$$l[5]=1$$

S	15	19	23	42	50	
l	1	2	3	1	2	



3.3 动态规划算法的典型应用

(6) 第六阶段 (10, 12, 15, 255, 1, 2)

$$j=1 \quad b[6]=2 \quad S[5]+1*2=52$$

$$j=2 \quad b[5]=2 \quad S[4]+2*2=46$$

$$j=3 \quad b[4]=8 \quad S[3]+3*8=47$$

$$j=4 \quad b[3]=8 \quad S[2]+4*8=51$$

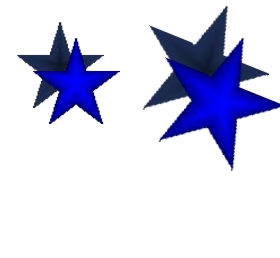
$$j=5 \quad b[2]=8 \quad S[1]+5*8=55$$

$$j=6 \quad b[1]=8 \quad S[0]+6*8=48$$

$$S[6]=\min\{52,46,47,51,55,48\}+11=57$$

$$l[6]=1$$

S	15	19	23	42	50	57
l	1	2	3	1	2	2



3.3 动态规划算法的典型应用

- 解的追踪

S	15	19	23	42	50	57
l	1	2	3	1	2	2

(1) 最少占用的位数: $S[6]=57$

(2) 分段, 从后向前的分段:

$l[6]=2$, 表示 $p_1—p_6$, 最后一个分段有2个像素
 $6-2=4$,

$l[4]=1$, 表示从 $p_1—p_4$, 最后一个分段有1个像素
 $4-1=3$

$l[3]=3$, 表示从 $p_1—p_3$, 最后一个分段有3个像素
 $3-3=0$, 结束



伪码

Compress (P, n)

1. $Lmax \leftarrow 256$; $header \leftarrow 11$; $S[0] \leftarrow 0$

2. for $i \leftarrow 1$ to n do

3. $b[i] \leftarrow \text{length}(P[i])$

4. $bmax \leftarrow b[i]$

5. $S[i] \leftarrow S[i-1] + bmax$

6. $l[i] \leftarrow 1$

7. for $j \leftarrow 2$ to $\min\{i, Lmax\}$ do

8. if $bmax < b[i-j+1]$

9. then $bmax \leftarrow b[i-j+1]$

10. if $S[i] > S[i-j] + j * bmax$

11. then $S[i] \leftarrow S[i-j] + j * bmax$

12. $l[i] \leftarrow j$

13. $S[i] \leftarrow S[i] + header$

子问题后
边界 i

最后
段长 j

找到更
好分段



追踪解

算法 Traceback (n, l)

输入：数组 l

输出：数组 C

1. $j \leftarrow 1$ // j 为正在追踪的段数
2. while $n \neq 0$ do
3. $C[j] \leftarrow l[n]$ 第 j 段长度
4. $n \leftarrow n - l[n]$
5. $j \leftarrow j + 1$

$C[j]$: 从后向前追踪的第 j 段的长度

时间复杂度: $O(n)$